

OSSP Skill Week 7 - I/O Redirection

Task 1: Input Redirection

Aim

To read input from a file by redirecting standard input using `dup2()` and to handle a missing input file.

Program

```
input_redirect.c > main(void)
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    int fd = open("input.txt", O_RDONLY);
    if (fd < 0) {
        perror("input.txt");
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        /* Child: standard input now reads from the file */
        if (dup2(fd, STDIN_FILENO) < 0) {
            perror("dup2");
            exit(1);
        }
        close(fd);

        execlp("cat", "cat", (char *)NULL);
        perror("exec");
        exit(1);
    }

    /* Parent: its own stdin is unchanged */
    close(fd);
    waitpid(pid, NULL, 0);

    printf("Input redirection completed successfully.\n");
    return 0;
}
```

Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % cd ~/SKILL_Week7
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % echo "Hello from input file" > input.txt
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % cat input.txt
Hello from input file
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % gcc input_redirect.c -o input_redirect
clang: error: no such file or directory: 'input_redirect.c'
clang: error: no input files
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % gcc '/Users/hrushika/Downloads/pjsdj/input_redirect.c' -o input_redirect
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % ./input_redirect
Hello from input file
Input redirection completed successfully.
❖ hrushika@Hrushikas-MacBook-Air SKILL_Week7 %
```

Result

Input redirection through stdin was implemented and tested successfully.

Task 2: Output Redirection

Aim

To redirect the standard output of a command to a file using `dup2()`.

Program

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6
7  int main(void)
8  {
9      int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
10     if (fd < 0) {
11         perror("output.txt");
12         return 1;
13     }
14
15     pid_t pid = fork();
16     if (pid < 0) {
17         perror("fork");
18         close(fd);
19         return 1;
20     }
21
22     if (pid == 0) {
23         /* Child: standard output now goes to the file */
24         if (dup2(fd, STDOUT_FILENO) < 0) {
25             perror("dup2");
26             exit(1);
27         }
28         close(fd);
29
30         execlp("echo", "echo", "Hello from output redirection", (char *)NULL);
31         perror("exec");
32         exit(1);
33     }
34
35     close(fd);
36     waitpid(pid, NULL, 0);
37
38     printf("Output redirection completed successfully.\n");
39     return 0;
40 }
```

Output

```
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % ./input_redirect
Hello from input file
Input redirection completed successfully.
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % gcc '/Users/hrushika/Downloads/pjsdj/output_redirect.c' -o output_redirect
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % ./output_redirect
Output redirection completed successfully.
● hrushika@Hrushikas-MacBook-Air SKILL_Week7 % cat output.txt
Hello from output redirection
○ hrushika@Hrushikas-MacBook-Air SKILL_Week7 %
```

Observation

The echo output did not appear on the terminal; it was written to `output.txt` instead. `O_CREAT` created the file with permissions `0644`, and `O_TRUNC` replaced any earlier contents.

Result

Output redirection was implemented successfully and the redirected data was verified.

Task 3: Append Redirection

Aim

To redirect output to a file in append mode while preserving its existing contents.

Program

```
append_redirect.c > main(void)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/wait.h>
6
7  int main(void)
8  {
9      /* O_APPEND makes every write go to the end of the file */
10     int fd = open("append.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
11     if (fd < 0) {
12         perror("append.txt");
13         return 1;
14     }
15
16     pid_t pid = fork();
17     if (pid < 0) {
18         perror("fork");
19         close(fd);
20         return 1;
21     }
22
23     if (pid == 0) {
24         if (dup2(fd, STDOUT_FILENO) < 0) {
25             perror("dup2");
26             exit(1);
27         }
28         close(fd);
29
30         execlp("echo", "echo", "New line appended to file", (char *)NULL);
31         perror("exec");
32         exit(1);
33     }
34
35     close(fd);
36     waitpid(pid, NULL, 0);
37
38     printf("Append operation completed successfully.\n");
39     return 0;
40 }
```

Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % echo "Existing data" > append.txt
● hrushika@Hrushikas-MacBook-Air pjsdj % cat append.txt
Existing data
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc append_redirect.c -o append_redirect
● hrushika@Hrushikas-MacBook-Air pjsdj % ./append_redirect
Append operation completed successfully.
● hrushika@Hrushikas-MacBook-Air pjsdj % cat append.txt
Existing data
New line appended to file
● hrushika@Hrushikas-MacBook-Air pjsdj % ./append_redirect
Append operation completed successfully.
● hrushika@Hrushikas-MacBook-Air pjsdj % cat append.txt
Existing data
New line appended to file
New line appended to file
○ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

Because the file was opened with `O_APPEND`, each run added "New line appended to file" after the existing text. After two runs the file held the original line followed by two appended lines.

Result

Append redirection was implemented successfully and the existing file data was preserved.

Task 4: Error Redirection

Aim

To redirect standard error to a file and capture the error messages produced by a failed command.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    int fd = open("error.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("error.txt");
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        /* Child: standard error now goes to the file */
        if (dup2(fd, STDERR_FILENO) < 0) {
            perror("dup2");
            exit(1);
        }
        close(fd);

        execlp("ls", "ls", "missing_file.txt", (char *)NULL);
        perror("exec");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Error redirection completed successfully.\n");
    return 0;
}
```

Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc error_redirect.c -o error_redirect
● hrushika@Hrushikas-MacBook-Air pjsdj % ./error_redirect
  Error redirection completed successfully.
● hrushika@Hrushikas-MacBook-Air pjsdj % cat error.txt
```

Observation

The error from ls did not appear on the terminal. It was written to error.txt, which contained "ls: cannot access 'missing_file.txt': No such file or directory".

Result

Standard error redirection was implemented successfully and the captured error message was verified.